

Trading / VAR

Data Architecture & Calculation Mechanism

*This prompt builds directly on the previously delivered "Trading / VAR Page: Functional Specification." Read together — this document covers **how parameters are stored** and **how the daily check is executed**, not the UI/page layout itself.*

Storage Format

Use **Parquet**, consistent with the existing data pipeline already in use for SPOT/RB price data. Store these files alongside the other source data files used by the pipeline — don't introduce a new storage location pattern for this feature alone.

Separate Files per Market

Two independent files, one per market:

- `var_params_spot.parquet`
- `var_params_rb.parquet`

Rationale: SPOT and RB are independent metrics with independent calibration cycles. Separate files avoid cross-contamination risk and keep each file simpler to reason about and debug.

History: Append-Only Log

Each file is an **append-only log** — every calibration run adds new rows; existing rows are never overwritten or deleted.

Schema (per row):

Column	Type	Description
calibration_id	string/int	Unique ID for this calibration run
calibration_timestamp	datetime	When this calibration was run
hour	string	H01–H24
volatility	float	Std. dev. of historical spread for this hour
lookback_window_days	int	Lookback window used for this calibration
accepted_limit_pln	float	Accepted Daily VaR limit (repeated across the 24 rows)
status	string	proposed or accepted
accepted_timestamp	datetime (nullable)	Set when user clicks Accept; null if still proposed

Why append-only, not overwrite: Preserves a full audit trail — see how volatility evolved calibration over calibration, compare lookback windows side by side, reconstruct what parameters were active on any past date. Data volume is tiny (24 rows/calibration, roughly weekly), so there's no performance concern — no partitioning or incremental-write complexity needed.

No separate "current parameters" file. The active parameter set is simply the most recent row(s) with status = accepted, per hour, filtered from the same history file. A single source of truth avoids "current" and "history" drifting out of sync.

Two Independent Procedures

Calibration logic and daily-check logic are **separate procedures** that only share the parameter file as their interface. Neither calls the other directly.

Procedure 1: Calibration (Section A)

- Triggered manually ("Recalculate SPOT/RB Parameters" buttons).
- Reads historical price data for the chosen lookback window.
- Computes per-hour volatility.
- Writes new rows with status = proposed.
- On "Accept", updates status to accepted and sets accepted_timestamp.
- Does not touch today's position data; does not compute any VAR — output is the parameter file only.

Procedure 2: Daily VAR Check ("Calculate VAR" button, Section B)

- Triggered manually — available on the VAR page and on the main Trading tab.
- Reads the latest accepted parameters (does not recalculate volatility).
- Reads today's open position per hour from the existing trades database (separately for SPOT/RB).

$$\text{Risk_hour} = |\text{Volume_MW}| \times \sigma_{\text{hour}} \times z \rightarrow \text{Daily_VAR} = \sum \text{Risk_hour} \rightarrow \% \text{ of limit}$$

- Returns a structured result (not a rendered view): % values for SPOT/RB + per-hour breakdown.

Separation of Calculation from Display

The output of Procedure 2 should be plain structured data. A separate rendering layer (whatever charting/gauge component is already used in the dashboard) takes that structure and draws the gauges/table. Keep these decoupled so calculation logic can be tested or reused independently of how it's displayed.

Notes for Implementation

- If exact file paths or naming conventions need to match an existing convention in the codebase, follow what's already used for other pipeline outputs — flag and ask if this isn't obvious from existing code.
- If anything in this document is ambiguous or conflicts with existing pipeline patterns, ask before proceeding rather than guessing.